



PROGRAMME DE FORMATION

Ingénierie DevOps

Construire, livrer et exploiter des applications avec une chaîne automatisée

81 HEURES

POUR Développeurs, administrateurs système et ingénieurs IT souhaitant évoluer vers des fonctions DevOps

PRÉPARE À HashiCorp Terraform Associate (004), thèmes principaux ·

GitHub Actions (GH-200), thèmes principaux · LPI DevOps Tools Engineer 2.0 (701-200), thèmes principaux



Mohamed Aziz BEN HAHA

mohamedazizbenhaha.netlify.app

Présentation

Ce programme suit une modification depuis le commit d'un développeur jusqu'à une application supervisée en production sur AWS. Les participants mettent en place la collaboration avec Git, construisent des pipelines d'intégration et de livraison continues avec Jenkins et GitHub Actions, ajoutent tests automatisés, seuils de qualité (quality gates) et artefacts versionnés, provisionnent l'infrastructure avec Terraform et configurent les serveurs avec Ansible, puis suivent le résultat grâce aux métriques, aux journaux et aux alertes. La même application est reprise dans toutes les phases, presque chaque chapitre se termine par un TP, et le programme s'achève par un projet final où les équipes assemblent toute la chaîne de livraison et la présentent.

Vue d'ensemble

#	PHASE	THÈMES	HEURES
01	Culture DevOps et collaboration	Principes DevOps, Git et méthodes de travail en équipe	9 h
02	Intégration et livraison continues	Jenkins, tests automatisés, SonarQube, Nexus, stratégies de déploiement, GitHub Actions	24 h
03	Infrastructure en code (IaC)	Provisionnement AWS avec Terraform, gestion de configuration avec Ansible	24 h
04	Supervision	Métriques et alertes Prometheus, tableaux de bord Grafana et journaux Loki	6 h
05	Projet final	Chaîne de livraison complète, présentée au groupe	9 h
	Réserve	Rattrapage, révision, TP supplémentaires	9 h
	Total		81 h

Informations clés

PUBLIC	Développeurs, administrateurs système et ingénieurs IT souhaitant évoluer vers des fonctions DevOps
PRÉREQUIS	Ligne de commande Linux et Bash (cours : Linux et scripting Bash) ; bases de Docker et Kubernetes (cours : Docker et Kubernetes). Ce cours prolonge Docker et Kubernetes sans le répéter.
DURÉE	81 h : 72 h de chapitres + 9 h de réserve
FORMAT	Cours courts suivis de TP guidés ; une même application reprise dans toutes les phases
ENVIRONNEMENT DE TP	Un portable par participant (16 Go de RAM minimum, Docker, VS Code) ; phases 1 et 2 en local, la phase 3 passe sur AWS. Un compte AWS de test par équipe, fourni par l'organisateur ; ressources détruites à la fin de chaque TP pour maîtriser les coûts
ÉVALUATION	Un TP validé à la fin de chaque phase et un projet final présenté au groupe ; attestation de formation pour les participants qui valident les deux
RÉSULTAT	Les participants savent faire passer une application du commit à la production via une chaîne automatisée : tests, seuils de qualité, artefacts versionnés, infrastructure en code, supervision et alertes

PHASE 01

9 H

Culture DevOps et collaboration

Le DevOps commence par la manière dont une équipe travaille, puis par la manière dont elle partage le code. Cette phase pose le vocabulaire et les indicateurs utilisés dans tout le programme, et permet à chaque participant de maîtriser Git et la collaboration par pull requests. La phase se termine par un TP validé : un dépôt d'équipe avec branches protégées, une pull request relue et une version taguée.

#	CHAPITRE	CONTENU	FORMAT	HEURES
1.1	Culture DevOps	CALMS, indicateurs DORA de performance de livraison (débit et instabilité, dont le taux de reprise), Agile et Kanban, cartographie de la chaîne de valeur, où l'automatisation est rentable	Cours + atelier	3 h
1.2	Fondamentaux de Git	Comment Git stocke l'historique (commits, arbres, références), init, clone, add, commit, log, diff, .gitignore, conventions de messages de commit	Cours + TP	1,5 h
1.3	Branches et fusions	Modèle de branches, fusions fast-forward et à trois voies, rebase, cherry-pick, résolution de conflits	Cours + TP	1,5 h
1.4	Dépôts distants et pull requests	fetch, pull, push, authentification SSH ou HTTPS, forks, origin et upstream, pull requests et revue de code	Cours + TP	1,5 h
1.5	Workflow Git en équipe	GitFlow ou trunk-based development, règles de protection des branches, tags de version, versionnage sémantique	Cours + TP	1,5 h

PHASE 02

24 H

Intégration et livraison continues

Les participants construisent le même pipeline deux fois : sur Jenkins, un serveur d'intégration continue auto-hébergé très utilisé, et sur GitHub Actions. Tests, seuils de qualité et dépôt d'artefacts garantissent que seules des versions testées et versionnées sont déployées, et la phase se termine par les stratégies de mise en production qui rendent un déploiement réversible. TP validé : l'application du cours construite, testée, analysée, publiée dans Nexus et déployée sur le cluster local par les deux pipelines.

#	CHAPITRE	CONTENU	FORMAT	HEURES
2.1	Bases de Jenkins	Installation, plugins, paramètres de sécurité, contrôleur et agents, types de jobs	Cours + TP	1,5 h
2.2	Déclencheurs et identifiants	Webhooks, scrutation du dépôt, builds planifiés, types d'identifiants et usage sécurisé	Cours + TP	1,5 h
2.3	Pipelines Jenkins	Jenkinsfile déclaratif, stages et steps, actions post, paramètres, variables d'environnement	Cours + TP	3 h
2.4	Passer Jenkins à l'échelle	Agents distribués, stages parallèles, bibliothèques partagées pour réutiliser le code des pipelines	Cours + TP	1,5 h
2.5	Tests automatisés dans le pipeline	Étapes de tests unitaires et d'intégration, rapports JUnit, arrêt au premier échec, environnements de test jetables en conteneurs	Cours + TP	1,5 h
2.6	Qualité du code avec SonarQube	Analyse statique, import de la couverture, profils qualité, seuils de qualité qui arrêtent le pipeline	Cours + TP	3 h
2.7	Gestion des artefacts avec Nexus	Dépôts hosted, proxy et group, Nexus comme registre Docker, versionnage, politiques de nettoyage, publication depuis le pipeline	Cours + TP	3 h
2.8	Déployer depuis Jenkins	Construire, étiqueter et publier les images dans Nexus, déployer sur Kubernetes avec Helm, promotion entre environnements	TP	1,5 h
2.9	Stratégies de déploiement et retour arrière	Déploiements progressifs, blue/green et canary, retour arrière, feature flags ; déploiement poussé ou GitOps (Argo CD, vu dans Docker et Kubernetes)	Cours + TP	1,5 h
2.10	GitHub Actions	Workflows, événements, jobs et steps, expressions et contextes, runners hébergés et auto-hébergés, builds matriciels, cache, artefacts	Cours + TP	3 h
2.11	Livraison sécurisée et réutilisation avec Actions	Secrets, environnements et validations obligatoires, fédération OIDC vers AWS sans clés permanentes, workflows réutilisables, actions personnalisées composites, JavaScript et Docker, politiques d'organisation et groupes de runners	Cours + TP	3 h

PHASE 03

24 h

Infrastructure en code (IaC)

L'infrastructure est décrite sous forme de code, relue comme du code et appliquée par le pipeline. Terraform crée l'environnement AWS de l'application du cours ; Ansible configure ensuite les serveurs qui hébergent les outils de l'équipe. Chaque outil se termine par un projet validé que le projet final réutilise.

#	CHAPITRE	CONTENU	FORMAT	HEURES
3.1	Bases de Terraform	Providers, HCL, ressources, data sources, graphe de dépendances, init, plan et apply, fmt et validate	Cours + TP	3 h
3.2	État et variables	Variables et validation, outputs, état et commandes terraform state, backend distant S3 et verrouillage, HCP Terraform	Cours + TP	3 h
3.3	Modules et environnements	Écrire et versionner des modules, Terraform Registry, workspaces, règles lifecycle, blocs import et moved, organisation du code	Cours + TP	3 h
3.4	Projet Terraform	Provisionner l'environnement du cours sur AWS : VPC, cluster EKS, base de données RDS et serveurs EC2 pour les outils	TP	3 h
3.5	Bases d'Ansible	Déclaratif ou impératif, dérive de configuration, architecture, inventaires statiques et dynamiques AWS, commandes ad hoc	Cours + TP	3 h
3.6	Playbooks	YAML, plays et tâches, modules principaux, handlers, conditions, boucles, idempotence	Cours + TP	3 h
3.7	Variables, templates et Vault	group_vars et host_vars, facts, templates et filtres Jinja2, Ansible Vault pour les secrets	Cours + TP	3 h
3.8	Rôles	Structure d'un rôle, valeurs par défaut, dépendances, Ansible Galaxy, conception de rôles réutilisables	Cours + TP	1,5 h
3.9	Projet Ansible	Configurer les serveurs d'outils EC2 (contrôleur et agents Jenkins, SonarQube, Nexus) avec des rôles, puis relancer pour prouver l'idempotence	TP	1,5 h

PHASE 04

6 H

Supervision

Une chaîne de livraison n'est terminée que lorsque l'équipe voit ce qui tourne en production. Les participants instrumentent l'application du cours, construisent des tableaux de bord à partir des métriques et des journaux, et acheminent les alertes vers les bonnes personnes. TP validé : un tableau de bord pour l'application du cours et une alerte qui se déclenche et atteint son point de contact.

#	CHAPITRE	CONTENU	FORMAT	HEURES
4.1	Métriques et alertes avec Prometheus	Modèle de métriques, exporters, instrumentation de l'application avec OpenTelemetry et les bibliothèques clientes Prometheus, PromQL, règles d'enregistrement et d'alerte, routage avec Alertmanager	Cours + TP	3 h
4.2	Tableaux de bord et journaux avec Grafana	Tableaux de bord, Loki pour les journaux, règles d'alerte Grafana, points de contact et politiques de notification, objectifs de niveau de service (SLO)	Cours + TP	3 h

PHASE 05

9 H

Projet final

Par équipes de deux ou trois, les participants assemblent tout le programme en une chaîne de livraison fonctionnelle pour une application fournie, puis la présentent comme ils le feraient devant un responsable technique.

#	CHAPITRE	CONTENU	FORMAT	HEURES
5.1	Cahier des charges et conception	Analyser le besoin, dessiner l'architecture cible, découper le travail en backlog, préparer le dépôt de l'équipe et l'environnement AWS à partir des projets de phase	Projet	3 h
5.2	Réalisation	Workflow Git ; intégration continue avec tests, seuil de qualité et artefacts versionnés ; infrastructure AWS avec Terraform ; serveurs d'outils configurés par Ansible ; déploiement sur EKS avec une stratégie de mise en production	Projet	3 h
5.3	Supervision et documentation	Métriques, journaux, tableaux de bord et alertes ; README, schéma d'architecture, runbook (procédures d'exploitation)	Projet	1,5 h
5.4	Démonstration et revue	Démonstration en direct d'une modification qui va du commit à la production, puis d'un retour arrière ; questions et retours	Projet	1,5 h

PROJET FINAL

Du commit à la production

Une modification poussée dans Git est testée, analysée, versionnée et déployée automatiquement sur une infrastructure AWS que l'équipe a provisionnée en code. Son état est visible sur des tableaux de bord avec alertes, et une mauvaise version peut être annulée.

OUTILS Git, GitHub, Jenkins ou GitHub Actions, SonarQube, Nexus, Terraform, Ansible, Docker, Kubernetes (EKS), Helm, Prometheus, Grafana, Loki

LIVRABLE Dépôt avec le code du pipeline et de l'infrastructure, schéma d'architecture, runbook, démonstration en direct